

Holt 2.0

Ignacio Díaz-Emparanza and Allin Cottrell

2026-09-14

1 Introduction

This package implements the forecasting method devised by C. C. Holt (1957), namely double exponential smoothing with level and trend and components. It requires time-series data. If you want triple exponential smoothing (with a seasonal component), as in Winters 1960, try the Winters package.

A prior version of this package (1.0) was written by Ignacio Díaz-Emparanza in 2015. This revision was written by Allin Cottrell, exploiting modern `hansl` syntax and adding functionality for automatic determination of the Holt smoothing parameters. See Section 7 below for help with updating scripts for use with version 2.

The filtering recursions and forecasting equation for Holt’s method are shown below. The notation is based on Hyndman (2021): ℓ denotes the level and b the trend or slope; α and β are the smoothing parameters.

$$\begin{aligned}\ell_t &= \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}) \\ b_t &= \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1} \\ \hat{y}_{t+h|t} &= \ell_t + hb_t\end{aligned}$$

Like most functions in `gretl`, `Holt` confines its operations to the sample range in force when the function is called. We’ll refer to the (1-based) start and end of this range as t_1 and t_2 , respectively.¹ In addition, let T denote the date of the last valid observation of the variable to be predicted, y .

There are two cases to be considered:

1. $T < t_2$: in this case there’s a ‘tail’ of observations for which values of y_t are not available. `Winters` will by default produce predictions for these observations.
2. $T = t_2$: there’s no tail for genuine out-of-sample prediction, but you can specify a number of observations, `ntest`, to reserve for pseudo-out-of-sample prediction. In that case the Holt training recursion will be terminated at observation $T - \text{ntest}$.

If you want case 1 but your dataset doesn’t contain any extra trailing observations it’s straightforward to add them using the `dataset addobs` command, or via the `Add observations` item under the `Data` menu in the graphical interface.

Conversely, if the dataset does contain extra trailing observations but you want case 2, specifying a positive value for `ntest` will result in the extra observations being ignored and pseudo-out-of-sample forecasts being produced.

2 The public functions

The primary function has the following signature:

```
function bundle Holt (const series y, const bundle specs[null])
```

¹This range can be set via the `smpl` command. If `smpl` has not been invoked since a dataset was opened t_1 will equal 1 and t_2 will equal the total number of observations.

The required argument `y` specifies the series to be processed. The second argument—which can be omitted, as indicated by the `null` modifier—may contain one or more of the following options:

- `theta` (matrix): if supplied, this must be a 2-vector holding the smoothing parameters $\theta = (\alpha, \beta)$, with each parameter in $[0, 1]$. If `parms` is not given these parameters are determined automatically.
- `ntest` (integer): specifies the number of observations at the end of the range of `y` values to reserve for testing via pseudo-out-of-sample forecasting. The default value is 0.
- `verbose` (integer): controls the verbosity of output, with levels 0, 1 (the default), 2 or 3.

The return value from `Holt` is a bundle containing the following elements:

- `theta`: a 2-vector holding the smoothing parameter values.
- `init`: a 2-vector of holding the initializer: level and trend.
- `ntest`: a non-negative integer indicating how many observations were reserved for pseudo-out-of-sample forecasting.
- `W`: a $T \times 4$ matrix. Column 1 holds the data passed as `y`; columns 2 and 3 hold the estimated level and trend; and column 4 holds forecasts. More details on this matrix are provided below.
- `RMSE_train`: the root mean squared error for the 1-step ahead forecasts made using the training data.

If `ntest` is non-zero, two additional items will be included in the bundle, namely the RMSE and Mean Absolute Error for the pseudo-out-of-sample forecasts, under the keys `RMSE_test` and `MAE_test`.

A simple plotting function is also supplied:

```
function void Holt_plot (bundle *b, int ptype)
```

The first argument should be a bundle obtained from `Holt` and the second argument should be either 1 for a plot of actual and forecast values, or 2 for a three-part plot of the level, trend and irregular components.

3 Command-line usage

We'll start with a simple example [FIXME FIND ONE!] replication of the Australian tourism example given by [Hyndman \(2021\)](#). Here's the `hansl` code:

```
open XXX.gdt
dataset addobs 12
bundle hb = Holt(XXX)
```

As an alternative to the above one might not add any observations but use the `ntest` option to get Winters to generate pseudo-out-of-sample forecasts:

```
open XXX.gdt #- -frompkg=Winters
bundle wb = Holt(y, _(ntest=8))
```

This will produce forecasts for 2016:1 to 2017:4.

4 Output details

Table 1 shows what is printed by `Holt` as a function of the `verbose` option. An example of the table of results displayed when `verbose` is 1 or greater is presented in the lower part of listing 1. The matrix `H` that's included in the bundle returned by `Holt` is basically a matrix version of this table. If you want to grab the columns of `H` as a list of series you can do something like

```
bundle hb = Holt(XXX)
list L = mat2list(hb.H)
list L print # show the members of L
```

Table 1: Effect of the `verbose` option

<i>verbose</i>	<i>cumulative effect</i>
0	Nothing is printed.
1	Parameters and initializer are reported, also a table of results (see below) and the SSE and RMSE for the training data.
2	If optimization is performed, partial information on progress of the iterative procedure is shown.
3	If optimization is performed, progress of the optimizer is shown in full.

5 GUI usage

When you install this package you should get the option of attaching it as the menu item `Variable/Filter` in `gretl`'s main window. Clicking this item brings up the dialog box shown in Figure 1. Hopefully the choices available in the dialog are fairly self-explanatory, but clicking the `Help` button will bring up a brief explanation of each control. On clicking `OK` you should get a window showing the results (Figure 2). The toolbar at the top of this window includes a `Graph` button which gives access to two types of plot, one showing actual and forecast y and one showing the level, trend and irregular components.

6 Technical details

This section offers some details on the rule-of-thumb initialization with which the Holt recursion is started, and the optimization that is carried out if `theta` is not given by the caller.

First, the rule-of-thumb is calculated as per the suggestion in Makridakis *et al.* (1998), which is also described in Hyndman (2010). Here τ denotes the first period in which y is observed.

- The initial level, $\ell_{\tau-1}$, is set to the mean of the m values $y_{\tau}, y_{\tau+1}, \dots, y_{\tau+m-1}$.
- The initial trend, $b_{\tau-1}$, is set to the mean of the first m seasonal differences:

$$m^{-1} \sum_{i=0}^{m-1} (y_{\tau+i+m} - y_{\tau+i})$$

- The initial seasonals are then set to the ratio of period-specific y_t to $\ell_{\tau-1}$ for the first m observations (multiplicative case) or the difference between y_t and $\ell_{\tau-1}$ (additive case).

As regards optimization, two cases arise.

Listing 1: Partial listing of results from the first example on page 3: Forecasts for 2018:1 and beyond are out-of-sample.

Holt double exponential smoothing
 Recursion runs from 1998:1 to 2017:4
 Out-of-sample forecast: 2018:1 to 2020:4

automatic parameter values:
 $\alpha = 0.246679$, $\beta = 0.122927$, $\gamma = 0$

estimated initializer:

level 9.7602
 trend -0.028109

training SSE = 13.4951, RMSE = 0.410718

	y	level	trend	season	forecast
1998:1	11.80604	9.79856	-0.01994	1.180433	11.48812
1998:2	9.27566	9.69731	-0.02994	0.981654	9.59922
1998:3	8.64249	9.54811	-0.04460	0.941049	9.09748
1998:4	9.29952	9.55357	-0.03844	0.958077	9.10509
...					
2017:1	12.40642	10.63437	0.08623	1.180433	12.60120
2017:2	10.47120	10.70735	0.08460	0.981654	10.52392
2017:3	10.49917	10.88197	0.09567	0.941049	10.15576
2017:4	11.21082	11.15617	0.1176140	0.958077	10.51742
2018:1					13.30794
2018:2					11.18241
2018:3					10.83055
2018:4					11.13920
...					

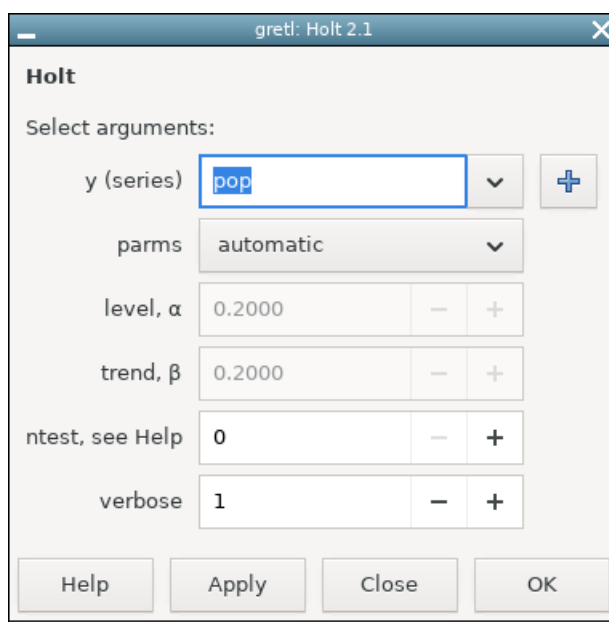


Figure 1: Holt dialog, under the Variable/Filter menu

- If `theta` is not specified, the parameter values and initializer are optimized jointly, with the criterion of minimizing the sum of squared forecast errors for the training range. The starting point for the parameters is $\alpha = \beta = 0.2$; the starting point for the initializer is as described above. Optimization is performed using the L-BFGS algorithm (see the chapter on Numerical methods in the *Gretl User's Guide* for details), with α and β constrained to lie in $[0, 1]$.
- If `theta` is given, the initializer is just the rule-of-thumb value; no optimization is performed. In principle the initializer could be optimized conditional on θ and that's a possible future refinement if it seems there's a real use case.

7 Updating from Holt 1.0

Listing 2 shows three cases drawing on the sample script for version 1.0 of the package along with the revised code required to produce similar results from version 2.0. Note that while in version 1.0 the `Holt` arguments are strictly positional, in version 2.0 they are in effect named, since they appear as elements of a `gretl` bundle (apart from the first required series argument). And the meaning of “default smoothing parameters” has changed: in version 1.1 that means using a fixed parameter vector, `FIXME` $\theta = (0.3, 0.1)$, while in version 2.0 it means that the parameters are automatically optimized, along with the initializer.

When θ is specified the two versions should produce identical forecasts: over the training range, $\hat{y}_{t|t-1}$, and over the testing or out-of-sample range, $\hat{y}_{T+h|T}$ where T is the last training observation and $h = 1, 2, \dots$ ² So in Case 1 we find that the testing results are the same for the two versions, namely `RMSE = XXX` and `MAE = XXX`.

In Case 2 of the listing we could have had version 2.0 replicate the results using the default θ from version 1.0, but it's more informative to compare that default with optimized results. The automatic θ in version 1.0 gives a slight improvement over Case 1: `RMSE = XXX`, `MAE = XXX`. The optimized parameters with 2.0 give a more substantial improvement: `RMSE = XXX`, `MAE = XXX`.

In Case 3, with automatic parameters and genuine out-of-sample forecasts, the forecasts will differ by version but we're obviously not in a position to offer forecast evaluation statistics.

The output data in the returned bundle also differ slightly across the versions of the package. In version 1.0 four series are included: the original series, the one-step ahead forecast, level-plus-trend, and seasonal. Version 2.0 gives a four-column matrix holding the original series, level, trend and forecast as in Hyndman (2021).

8 Change log

- `v2.0`: Initial release of substantially modified version.

References

- Holt, C. C. (1957) 'Forecasting seasonals and trends by exponentially weighted moving averages'. ONR Memorandum, Vol. 52, Carnegie Institute of Technology. URL <https://doi.org/10.1016/j.ijforecast.2003.09.015>.
- Hyndman, R. (2010) 'Initializing the Holt-Winters method'. <https://robjhyndman.com/hyndsight/hw-initialization/>. Accessed: 2026-03-09.
- _____ (2021) 'Exponential smoothing: seasonality'. <https://otexts.com/fpp3/holt-winters.html>. Accessed: 2026-03-09.

²`FIXME`. With one qualification: following Hyndman's procedure the first training forecast in version 2.0 is for the first observation of y , while in version 1.0 the forecasts start m periods later.

Makridakis, S., S. C. Wheelwright and R. Hyndman (1998) *Forecasting: Methods and Applications*, New York: Wiley.

Winters, P. R. (1960) 'Forecasting sales by exponentially weighted moving averages', *Management Science* 6(3): 324–342. URL <https://doi.org/10.1287/mnsc.6.3.324>.

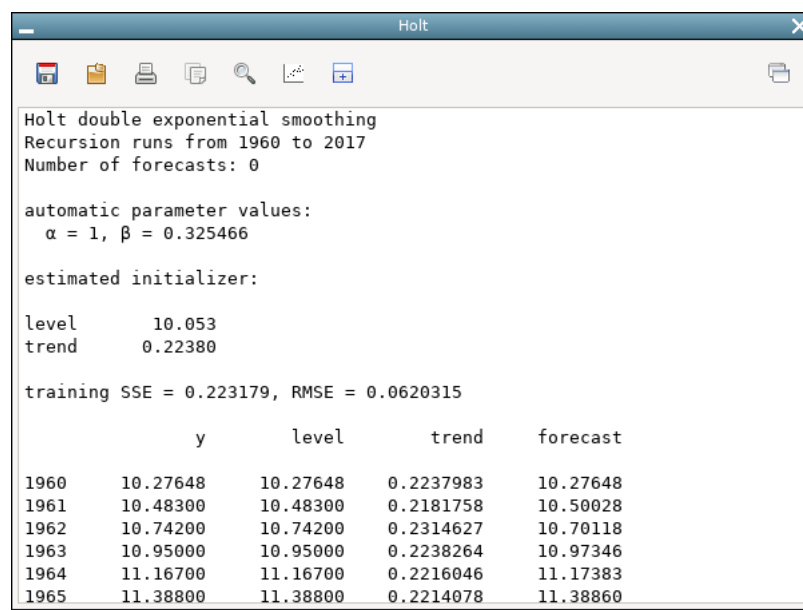


Figure 2: Holt output window

Listing 2: Winters 1.1 examples with corresponding 2.0 code

Common initial code

```
include Holt.gfn
open data9-3.gdt
```

Case 1: Multiplicative, 4 testing forecasts, specified parameters

```
# version 1.1
bundle BW1 = Holt(reskwh, 1, 0.5, 0.1, 0.7, 4)
series reskwh_W_mul = BW.reskwh_W_mul

# version 2.0
bundle BW1 = Holt(reskwh, _(theta = {0.5, 0.1, 0.7}, ntest = 4))
series reskwh_W_mul = BW.W[,5]
```

Case 2: Multiplicative, 4 testing forecasts, automatic parameters

```
# version 1.1
bundle BW2 = Holt(reskwh,1,,,4)
series reskwh_W_mul = BW2.reskwh_W_mul

# version 2.0
bundle BW2 = Holt(reskwh, _(ntest=4))
series reskwh_W_mul = BW2.W[,5]
```

Case 3: Additive, 4 out-of-sample forecasts, automatic parameters

```
# version 1.1
dataset addobs 4
bundle BW3 = Holt(reskwh, 0)
series reskwh_W_add = BW3.reskwh_W_add

# version 2.0
dataset addobs 4
bundle BW3 = Holt(reskwh, _(additive=1))
series reskwh_W_add = BW3.W[,5]
```
